



Multimodal API

SAKURA AI Engine Practical Exercise Module 3

<https://www.sakura.ad.jp/>

DATE

2026.8

COMPANY

SAKURA internet Inc.

SERVICE

SAKURA AI Engine

VERSION

2026.8

Today's Agenda

SAKURA AI Engine Practical Exercise consists of four sessions, 1 through 4.

Module 1

SAKURA AI Engine Practical Exercise

Log in, select a model, issue an Account Token, run inference, and build a RAG chatbot using a document.

Module 2

SAKURA AI Engine Practical Exercise 2

Converting audio to text (transcription): Comparing accuracy in English and Thai to summarize.

Module 3

SAKURA AI Engine Practical Exercise 3

Combine image and text input to design AI output that supports business decisions.

★ You are here

Module 4

SAKURA AI Engine Practical Exercise 4

Embed image and text AI calls into a web app and run it in a browser.

Goals of this class

Going a step beyond "asking questions to AI," we create AI that reads our materials and has them answer.

1 Understanding how multimodal APIs work

Understanding the basic composition of the Vision-Language model, which integrates text and images for inference.

2 Send an image to the API using a public URL.

Specify images via URL and call multimodal APIs via text

3 Send a local image using Base64 encoding.

Learn how to directly embed private business image files into the API

4 Learning Limitations and Solutions for Image Transmission

If these four are achieved, AI will take a step closer from being a "user" to a "person embedding it."

Timetable (90 minutes)

In this 90-minute session, you will learn the fundamentals of multimodal AI and gain hands-on experience with AI-powered image recognition.

Time	Contents
0 - 10	What is Multimodal AI? / Multimodal Models in SAKURA AI Engine
10 - 25	Hands-On 1 Multimodal Model Operation Verification (Qwen3-VL Response Confirmation)
25 - 45	Hands-On 2 Image URL / Image Recognition API Using Base64
45 - 60	Hands-On 3 Base64 Image Processing Using JSON File Format
60 -75	Hands-On 4: Hands-On Development and Considerations
75 - 90	Q&A and Summary

How This Class Works

Use the supporting website during the exercises. Keep the slides open to see what you are working on.

Slides and Supporting Website

- Slides: Explanations and the current task
- Website: Detailed steps, commands, and extra challenges
- Refer to the website while working

Required and Optional

- Each hands-on exercise has a required task and an optional challenge
- After completing the required task, continue to the optional challenge
- Work at your own pace

If You Get Stuck

- Search for the error message in Troubleshooting
- Ask a nearby classmate for help
- If the issue remains, ask the instructor or a TA

Overview (1) (0-10 minutes)

What is Multimodal AI?

Modality and Multimodal

✓ What is Modality?

A modality is a type of data, such as text, images, audio, video, or sensor data.

✓ What is Multimodal AI?

Multimodal AI is an AI technology that can process and integrate multiple types of data simultaneously.



Differences Between Single-Modal AI and Multimodal AI

- ✓ With single-modal AI, only one type of data could be handled—just text or images.
- ✓ Multimodal AI can process multiple types of data by combining them.
- ✓ By integrating and processing multiple modalities, deeper understanding and reasoning can be achieved.

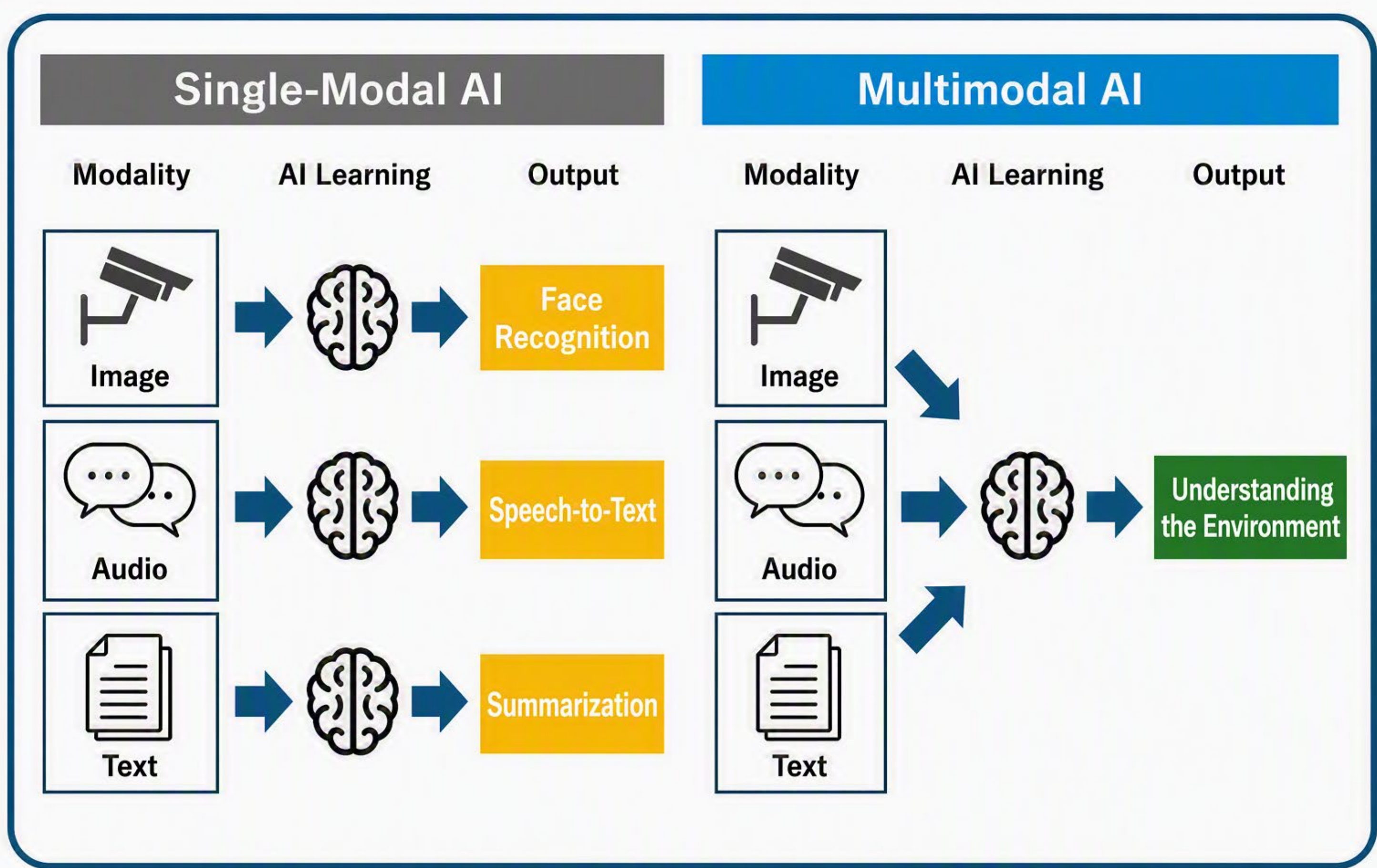
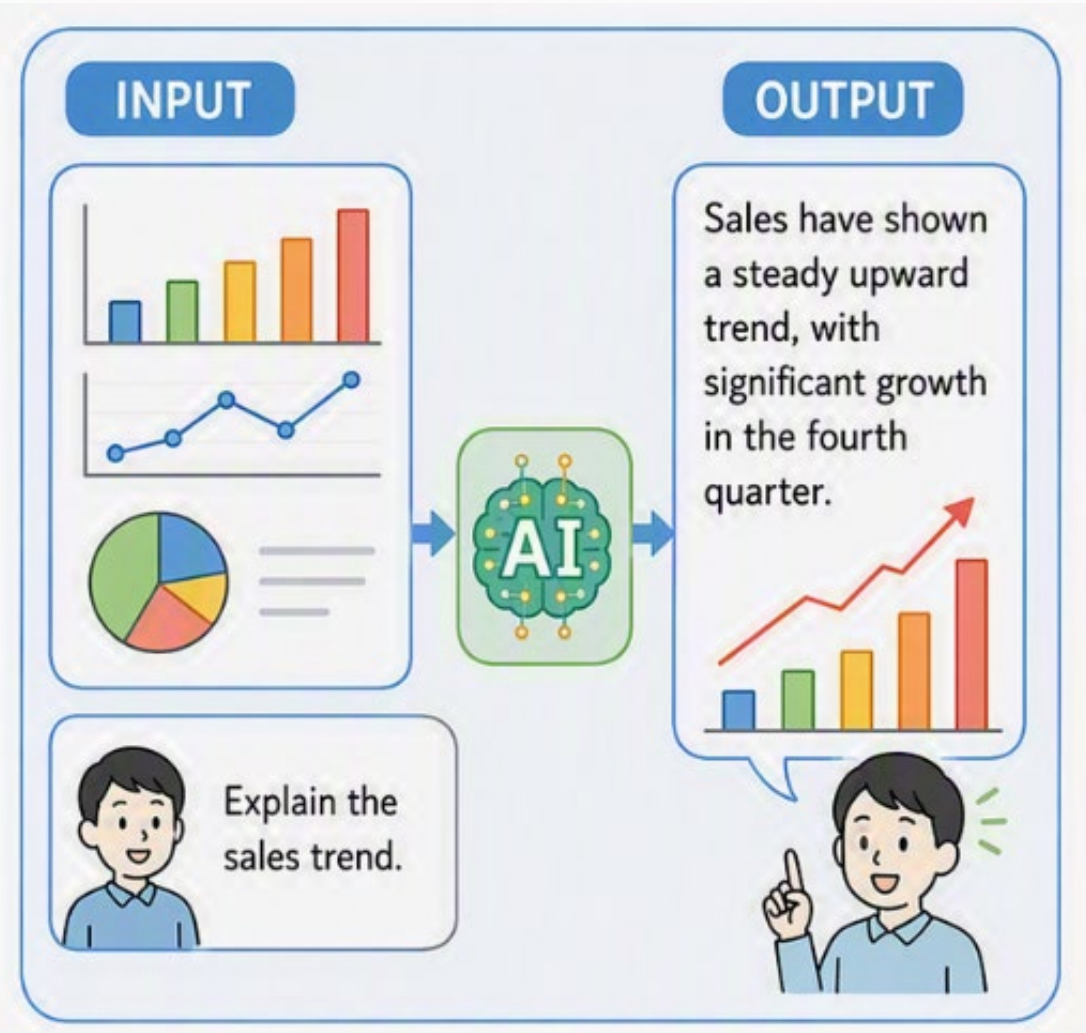


Image: <https://ai.sakura.ad.jp/column/what-is-multimodalai/>

Examples of Multimodal AI Applications

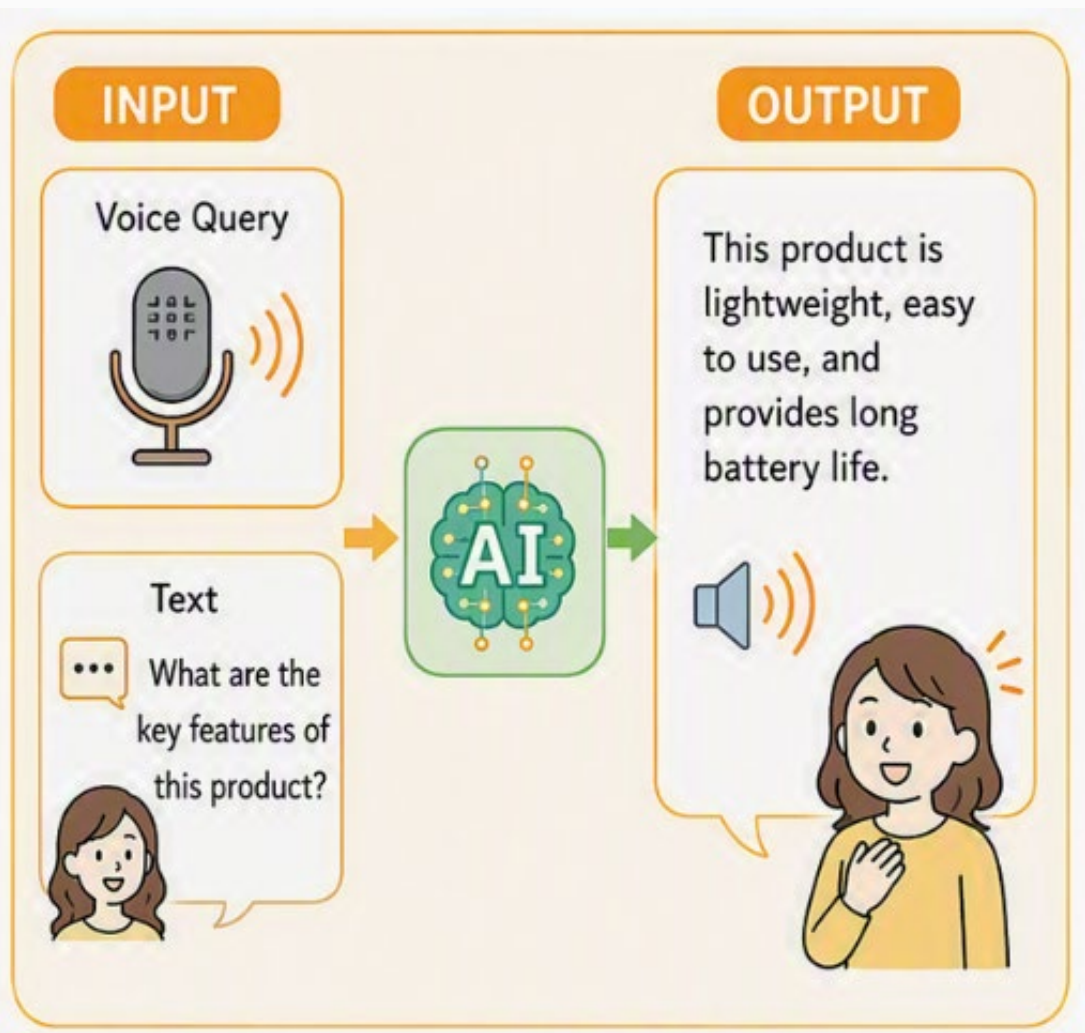
■ Images and Text

- Summary and explanation of figures and tables
- Visual QA



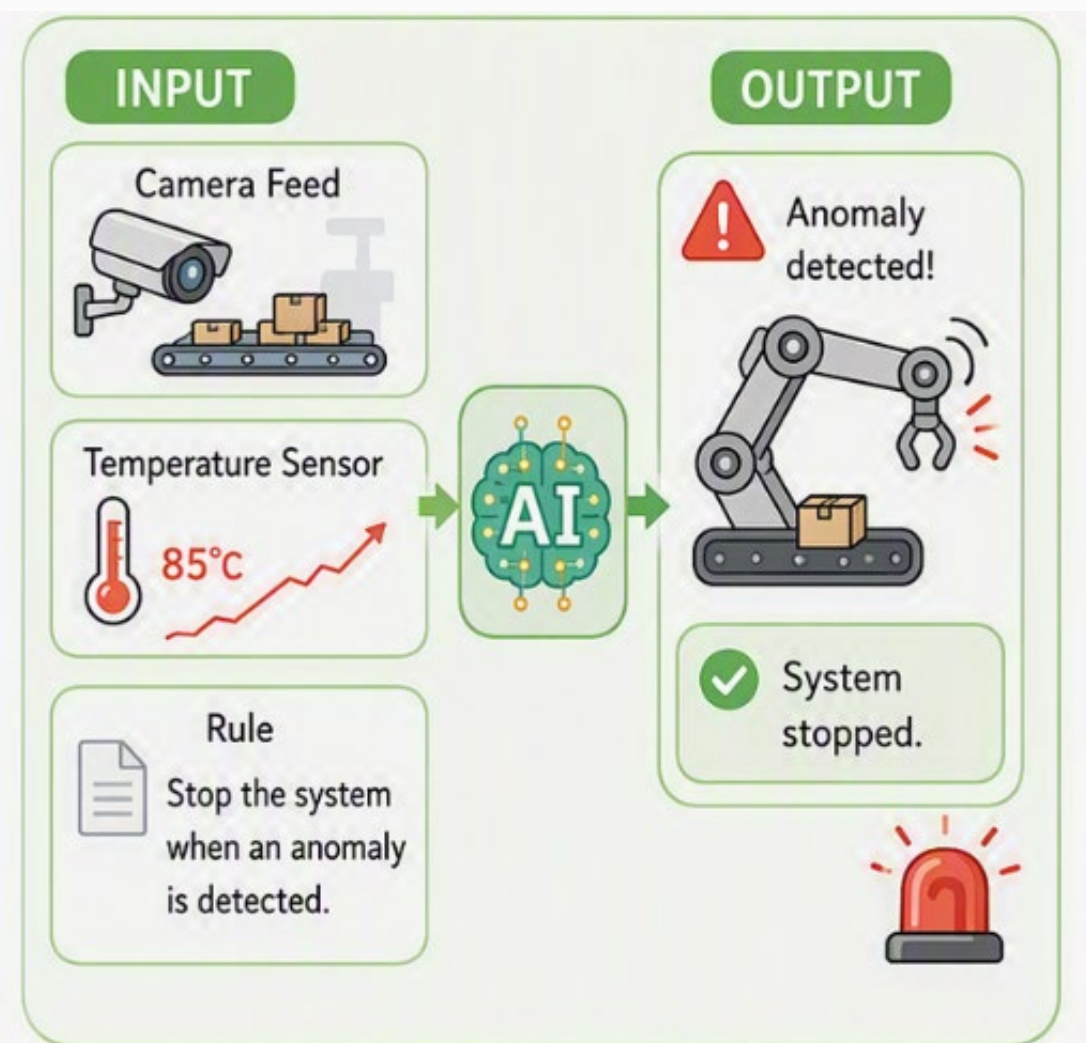
■ Audio and Text

- Natural Voice Interaction
- Contextual explanation



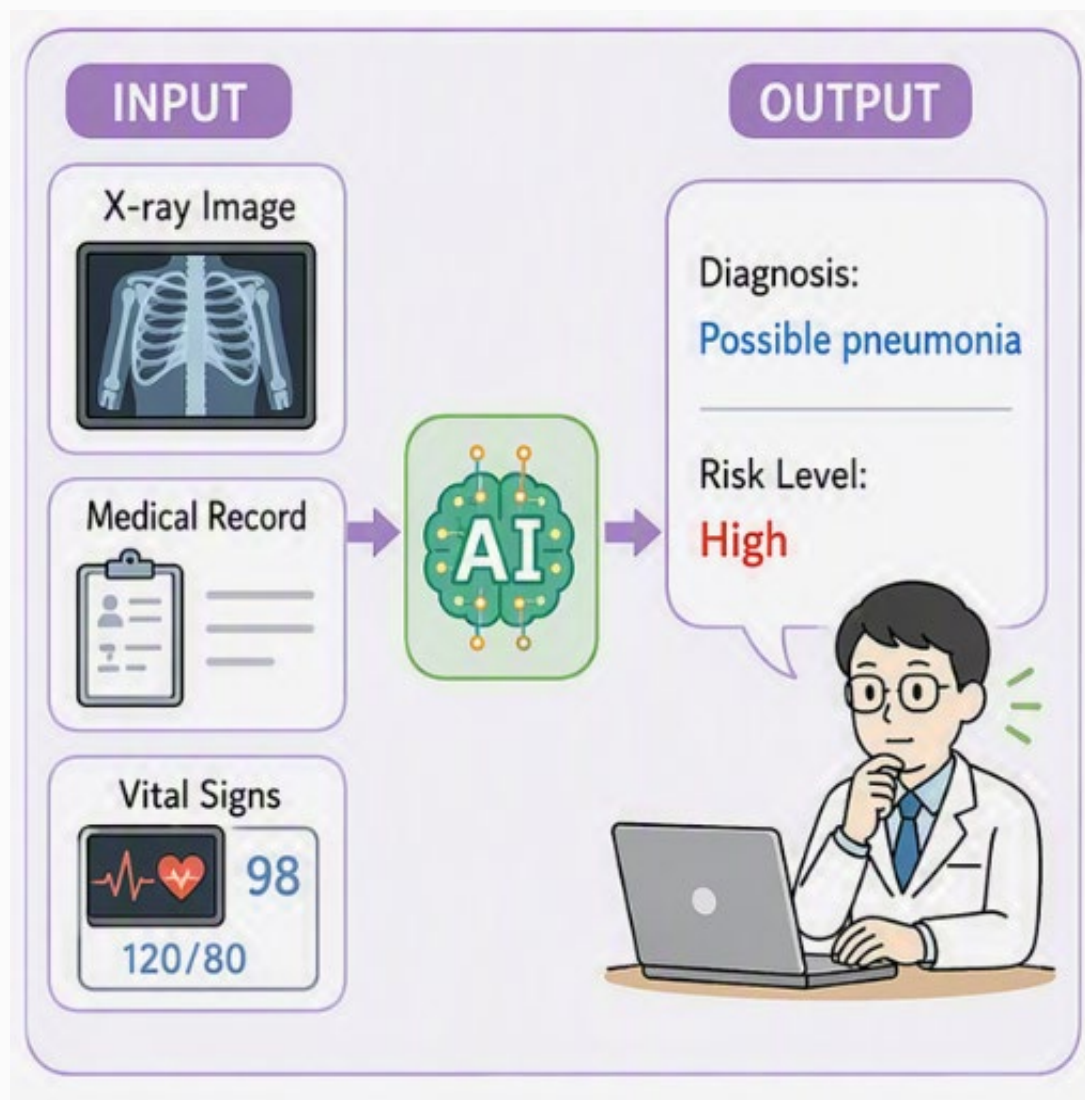
■ Sensor Information and Text

- Situation Analysis from Camera and Sensor Data
- Robot Control



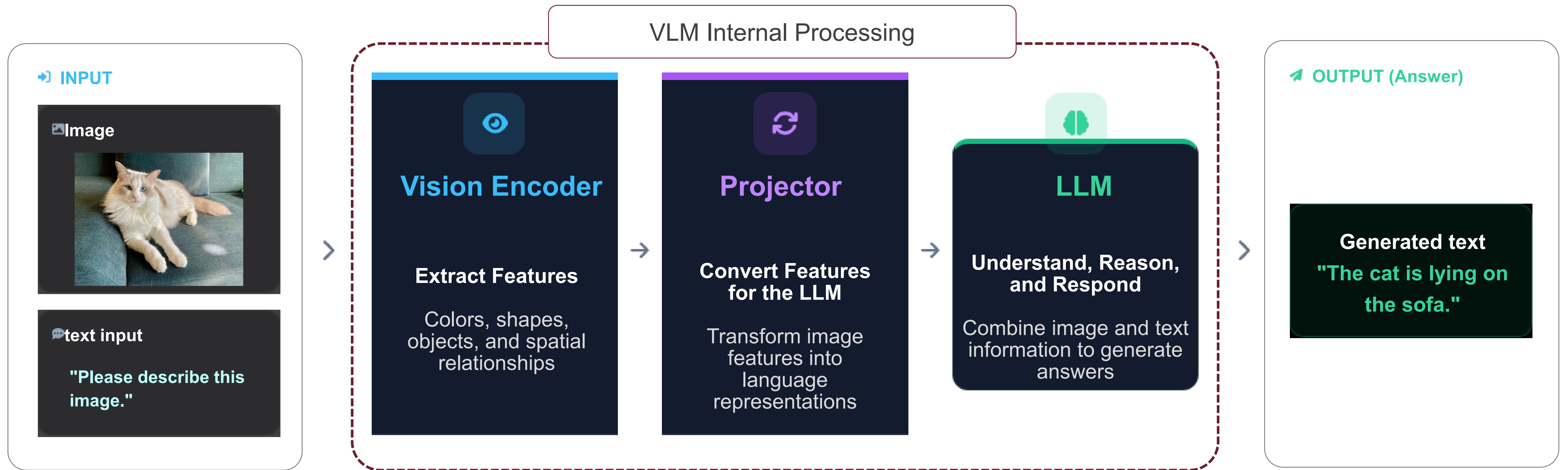
■ Decision Support Based on Multiple Information Sources

- Integrated Data Analysis for Healthcare, Manufacturing, and Security



How does AI "understand" images?

How Visual Language Models (VLMs) Work

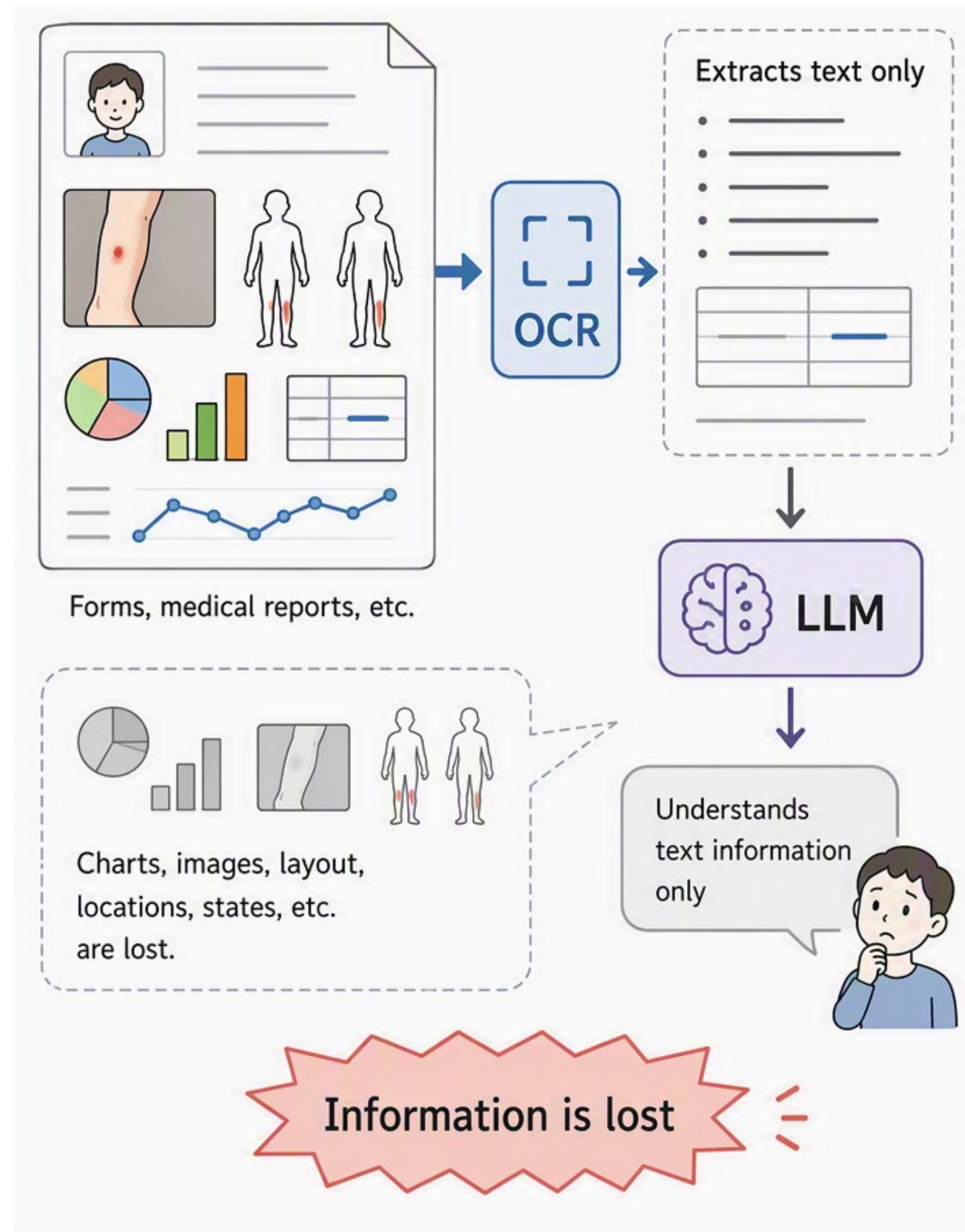


A VLM combines visual understanding with language understanding

Differences from "OCR+LLM"

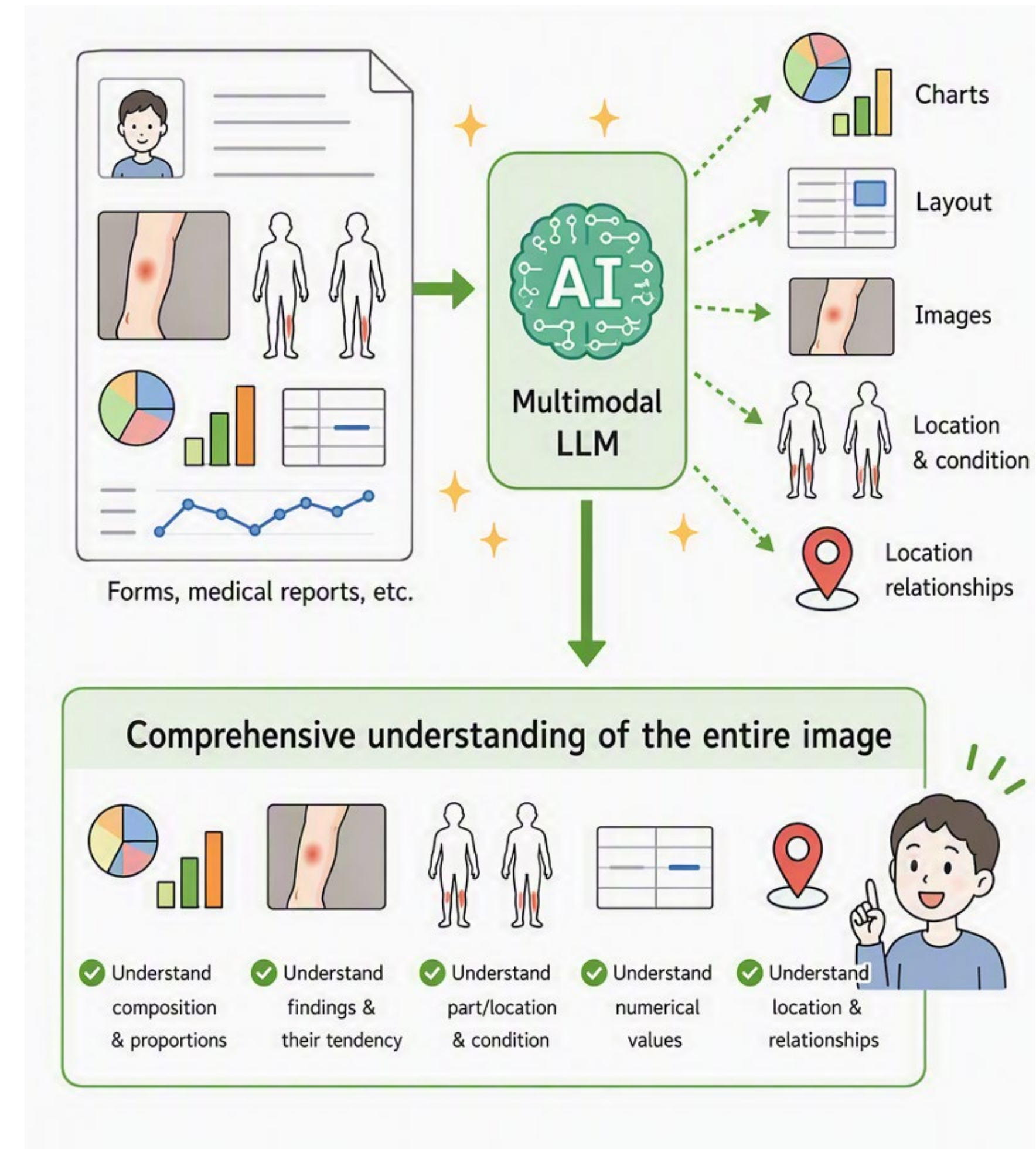
■ OCR (Optical Character Recognition)

- Find text from images and convert them to text



■ VLM

- Analyzes images and text together to understand meaning and structure

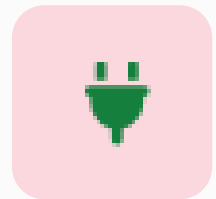


Hands-On 1

Exploring a Multimodal Model

(Response Evaluation using Qwen3-VL-30B-A3B-Instruct)

SAKURA AI Engine and Public Preview Models



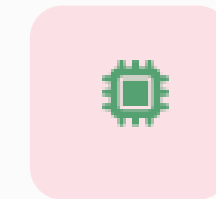
Multimodal models provided by SAKURA AI Engine

- Offered as a public preview model

What is the Public Preview Model?

An early provision model aimed at collecting user feedback

⚠ Please note that specifications may change or service may be discontinued without prior notice.



Model used in this session: Qwen3-VL-30B-A3B-Instruct

Vision-Language model in the Qwen series that can process images and text.

- VL (Vision-Language): Processes images and text together
- Instruct: Follows user instructions and prompts
- 30B-A3B: Indicates the model architecture and size (Mixture of Experts)
 - Total parameters about 30B, active parameters during inference about 3B

Check the model in the control panel

<https://secure.sakura.ad.jp/ai/models/text>

 さくらのAI Engine

Available models

Account token

Document (RAG)

Usage amount

Playground

preview/Phi-4-multimodal-instruct	multimodal	☐ End of offer
preview/Qwen3-0.6B-cpu	Public Preview CPU	☒ available
preview/Qwen3-VL-30B-A3B-Instruct	Public Preview multimodal	☒ available
preview/Qwen3.6-35B-A3B	Public Preview multimodal coding	☒ available
+1 ▲		
preview/gemma-4-31B-it	Public Preview multimodal coding ResponsesAPI support MessagesAPI support	☒ available
+3 ▲		

STEP 1 | Set Up Environment Variables

- Make sure that API authentication token has been set as an environment variable
- Use the account token issued in advance

```
export AI_ENGINE_TOKEN="Account token issued"
```

1

Setting up account tokens

Set account tokens as environment variables

STEP 2 | Chat with Text Only

First, test the model with a simple text-only conversation.

```
curl -X POST "https://api.ai.sakura.ad.jp/v1/chat/completions" ¥
-H "Authorization: Bearer ${AI_ENGINE_TOKEN}" ¥
-H "Content-Type: application/json" ¥
-d '{
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "Hello"
        }
      ]
    }
  ],
  "temperature": 0.7,
  "max_tokens": 1000,
  "stream": false
}' | json_pp
```



Endpoint

Send to chat/Completions



model

Name of model to be used



text

Send text-only input



Generation Settings

Configure parameters such as temperature

json_pp

Converts JSON into a clear, readable format (bundled with Perl)

Reference: Display API Responses in a Readable Format

- JSON responses are difficult to read on a single line
- **jq** is a popular JSON formatter (ex: `curl ... | jq`)
- Since sudo privileges are not available in this environment, we use `json_pp` instead
- No additional installation is required for **json_pp** (`curl ... | json_pp`)

Without formatting

```
{"id":"chatcmpl-a54d939bf14fb0c0","object":"chat.completion","created":1788472409,"model":"preview/Qwen3-VL-30B-A3B-Instruct","choices":[{"index":0,"message":{"role":"assistant","content":"Hello! How can I assist you today? 😊","refusal":null,"annotations":null,"audio":null,"function_call":null,"reasoning":null},"logprobs":null,"finish_reason":"stop","stop_reason":null,"token_ids":null,"routed_experts":null}],"service_tier":null,"system_fingerprint":"vllm-0.26.0-tp2-6ff63b19","usage":{"prompt_tokens":9,"total_tokens":21,"completion_tokens":12,"prompt_tokens_details":null,"prompt_logprobs":null,"prompt_token_ids":null,"prompt_text":null,"kv_transfer_params":null,"ec_transfer_params":null,"metrics":null}yo-shiba
```

With formatting

```
{
  "id": "chatcmpl-8c99b33a8e30759a",
  "object": "chat.completion",
  "created": 1788472478,
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! How can I assist you today? 😊",
        "refusal": null,
        "annotations": null,

```

[Ref] Meaning of Request Parameter

Parameter	Meaning
model	Name of the AI model to use
messages	Conversation content, including roles and messages
role	Speaker role: user for user input; system for AI instructions
content	Message text to send. Edit this field.
temperature	Controls response diversity. Lower is more consistent; higher is more varied.
max_tokens	Maximum number of tokens in the generated response
stream	Response mode: streaming or non-streaming

STEP 3 | Review the Response

Check the response text, finish reason, and token usage.

```
{
  "choices": [
    {
      "finish_reason": "stop",
      "index": 0,
      "message": {
        "content": "Hello! How can I assist you today? 😊",
        "role": "assistant"
      }
    }
  ],
  "created": 1788379031,
  "id": "chatcmp1-825f6428127b01ea",
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "object": "chat.completion",
  "system_fingerprint": "vllm-0.26.0-tp2-6ff63b19",
  "usage": {
    "completion_tokens": 12,
    "prompt_tokens": 9,
    "total_tokens": 21
  }
}
```



finish_reason

stop indicates the response completed normally



content

Main body of the response



usage

Number of tokens consumed (affects cost).

• 400 Bad Request

A request parameter may be missing or invalid.

• 401 Unauthorized

Missing token specification.

Double-check with echo \$AI_ENGINE_TOKEN.

Hands-On 2

Image Recognition with Image URLs and Base64 Encoding

Two methods of image transmission

Technique 1

URL method

- Provide the URL of a publicly accessible image (https://...) in the request.

Advantage: Request body is lightweight at only a few KB

Restrictions: Cannot access local files or internal images that are not available on the Internet

```
"type": "image_url",
      "image_url": {
        "url": "https://xxxx/sakura-office.jpg"
      }
```

Technique 2

Base64 Encoding Method

- Convert a local image file into a Base64-encoded text string before sending it.
- Effective when handling private images

Advantage: Useful when working with private or local images.

Note: Base64 encoding increases the amount of data transferred

```
"type": "image_url",
      "image_url": {
        "url": "data:image/jpeg;
base64,'$(base64 -w 0 flower.jpg)'"
      }
```

"-w 0" means "Do not break lines midway through the output of Base64."

STEP 1 | Send an Image URL to the API

Send an image together with a prompt such as “Please describe this image.”

```
curl -X POST "https://api.ai.sakura.ad.jp/v1/chat/completions" \
-H "Authorization: Bearer ${AI_ENGINE_TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "Please describe this image briefly."
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "https://s3.tky01.sakurastorage.jp/ai-kentei/sakura-office.jpg"
          }
        }
      ]
    }
  ],
  "temperature": 0.7,
  "max_tokens": 2000,
  "stream": false
}' | json_pp
```

Here is the image to be sent



Text

Send a question or instruction



Image URL

Specify the public URL for images stored in SAKURA's Object Storage

Multiple inputs can be included in the same message

STEP 2 | Review the Response

```
{
  "choices": [
    {
      "finish_reason": "stop",
      "index": 0,
      "message": {
        "content": "This image shows a modern, bright office lounge or break area with large windows offering a city view. The space features a prominent red perforated metal screen with the ¥\"SAKURA Internet¥\" logo, wooden tables and chairs, built-in benches, and large planters with greenery. The overall design is clean and contemporary.",
        "role": "assistant"
      }
    }
  ],
  "created": 1788379635,
  "id": "chatcmpl-924c085a36a9c1af",
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "object": "chat.completion",
  "system_fingerprint": "vllm-0.26.0-tp2-6ff63b19",
  "usage": {
    "completion_tokens": 66,
    "prompt_tokens": 1191,
    "total_tokens": 1257
  }
}
```



content

The image is described in natural language.

Image recognition can be enabled simply by adding an image to the input without changing the API usage significantly.

STEP3 | Download the Image to Your Local PC

Download the image from the URL below and store it in the specified folder

画像URL <https://s3.tky01.sakurastorage.jp/ai-kentei/flower.jpg>

Here is the image to store ↓



```
mkdir ~/03_Multimodal
```

1

Create a directory named 03_Multimodal

```
cd ~/03_Multimodal
```

2

Move to 03_Multimodal

```
wget "https://s3.tky01.sakurastorage.jp/ai-kentei/flower.jpg" -O flower.jpg
```

3

Download the image file

```
ls -lh flower.jpg
```

4

Confirm that image files has been stored

→

```
yo-shibata@~:~/03_Multimodal$ ls -lh flower.jpg  
-rw-r--r-- 1 yo-shibata yo-shibata 53K Feb  4 2026 flower.jpg
```

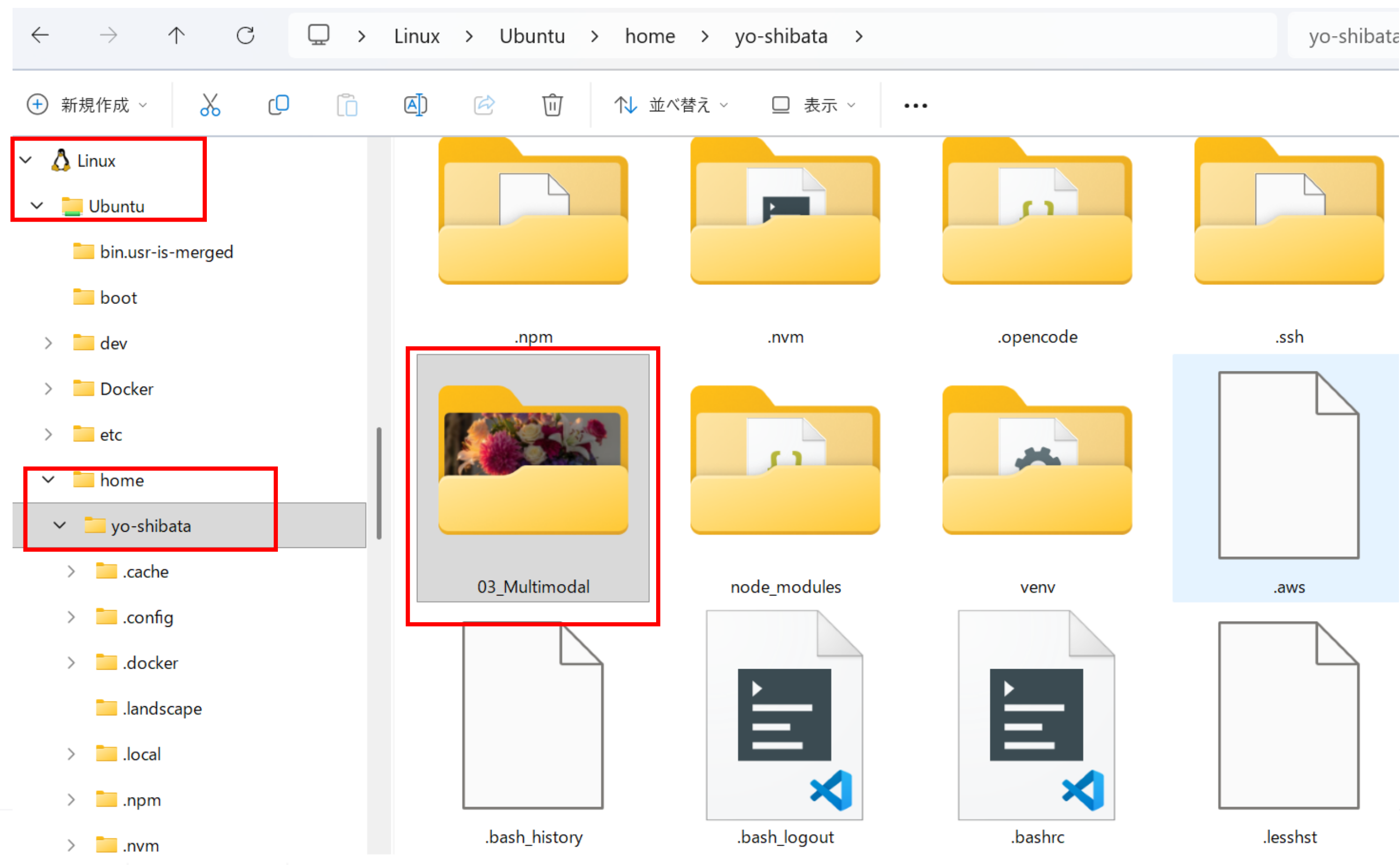
STEP4 | Verify the Image Saved Locally

On Windows PC, the downloaded image is stored in the following folder within File Explorer.

Linux > Ubuntu > home > {username} > 03_Multimodal

Alternatively, enter the following path directly:

→ ¥¥wsl.localhost¥Ubuntu¥home¥{username} ¥03_Multimodal



The stored image ↓



STEP5 | Send a Local Image to the API as Base64

```
curl -X POST "https://api.ai.sakura.ad.jp/v1/chat/completions" \
-H "Authorization: Bearer ${AI_ENGINE_TOKEN}" \
-H "Content-Type: application/json" \
-d '{
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        {
          "type": "text",
          "text": "Please describe this image briefly."
        },
        {
          "type": "image_url",
          "image_url": {
            "url": "data:image/jpeg;base64,'$(base64 -w 0 flower.jpg)'"
          }
        }
      ]
    }
  ],
  "temperature": 0.7,
  "max_tokens": 2000,
  "stream": false
}' \
| json_pp
```



Image URL

Embed a local image in the URL using Base64 encoding

Private or local images can be included directly in the request using Base64 encoding.

STEP 6 | See the response

```
{
  "choices": [
    {
      "finish_reason": "stop",
      "index": 0,
      "message": {
        "content": "A vibrant bouquet of flowers, including pink dahlias, purple hydrangeas, white and pink roses, and lilies, is arranged in a translucent, frosted glass vase. The vase sits on a wooden table, illuminated by a warm, golden light from behind, creating a soft glow.",
        "role": "assistant"
      }
    }
  ],
  "created": 1788379996,
  "id": "chatcmpl-a2fdb9c6ef444aec",
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "object": "chat.completion",
  "system_fingerprint": "vllm-0.26.0-tp2-6ff63b19",
  "usage": {
    "completion_tokens": 60,
    "prompt_tokens": 170,
    "total_tokens": 230
  }
}
```



content

The image is described in natural language.

Progress Check

Checking Progress

Before proceeding, please make sure you have completed the following steps.

- ✓ **Received a response to a text-only request using Qwen3-VL-30B-A3B-Instruct**
- ✓ **Sent an image from external storage using an image URL and received a response describing the image**
- ✓ **Downloaded an image from the specified URL to your local PC**
- ✓ **Sent a locally stored image as Base64-encoded data and received a response describing the image**

Need Help?

If you encounter any issues, please ask an instructor or TA for assistance.

Hands-On 3

Base64 Image Processing Using JSON Files

⚠ Base64 Image Processing and Command-Line Length Limits

⚠ Error due to command argument limiting

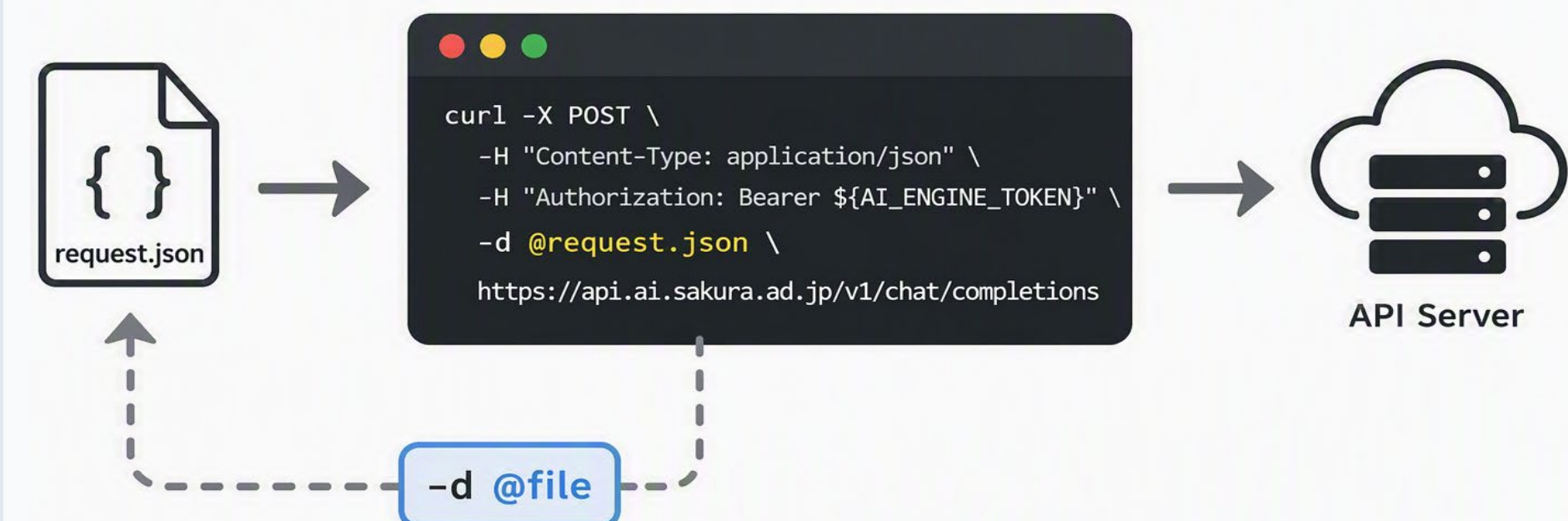
- Converting a high-resolution image to Base64 can generate a very large text string (a few MBs), which may exceed the operating system's command-line limit when sent directly with curl.
- Error name: **Argument list too long** (OS ARG_MAX exceeded)
- Estimated limit: Approximately 120,000 characters generated in a practical training environment

Command to check the number of characters after conversion

```
wc -c image_base64.txt
```

✓ Solution: Specify JSON file submission

Save the request body to a JSON file (request.json) and submit it using the `curl -d @file` option.

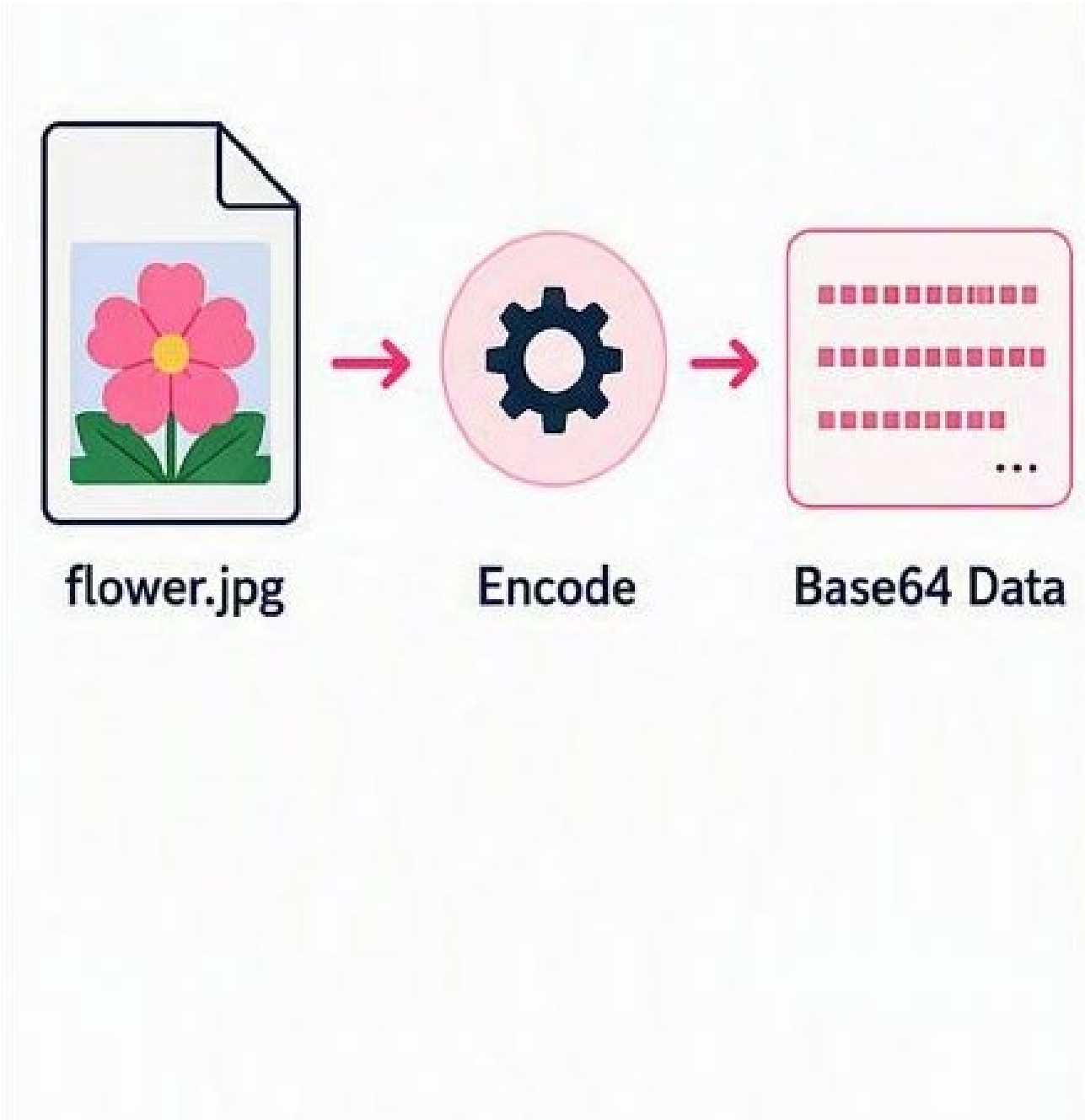


Workflow for Sending Local Images

STEP 1:

Convert Image to Base64

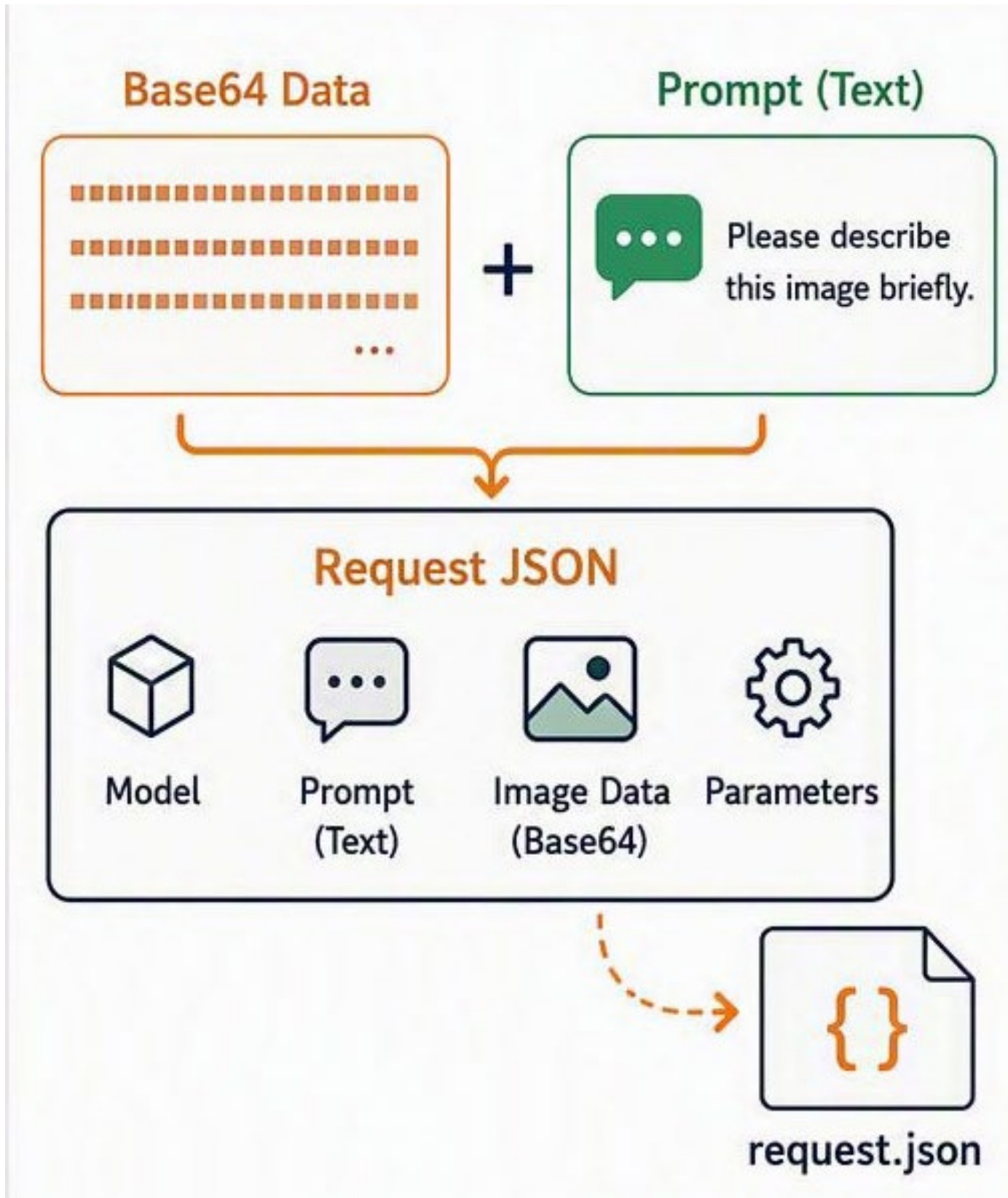
Convert local images ('sample.jpg') to Base64 strings (Data URI format)



STEP 2:

Create a JSON Request File

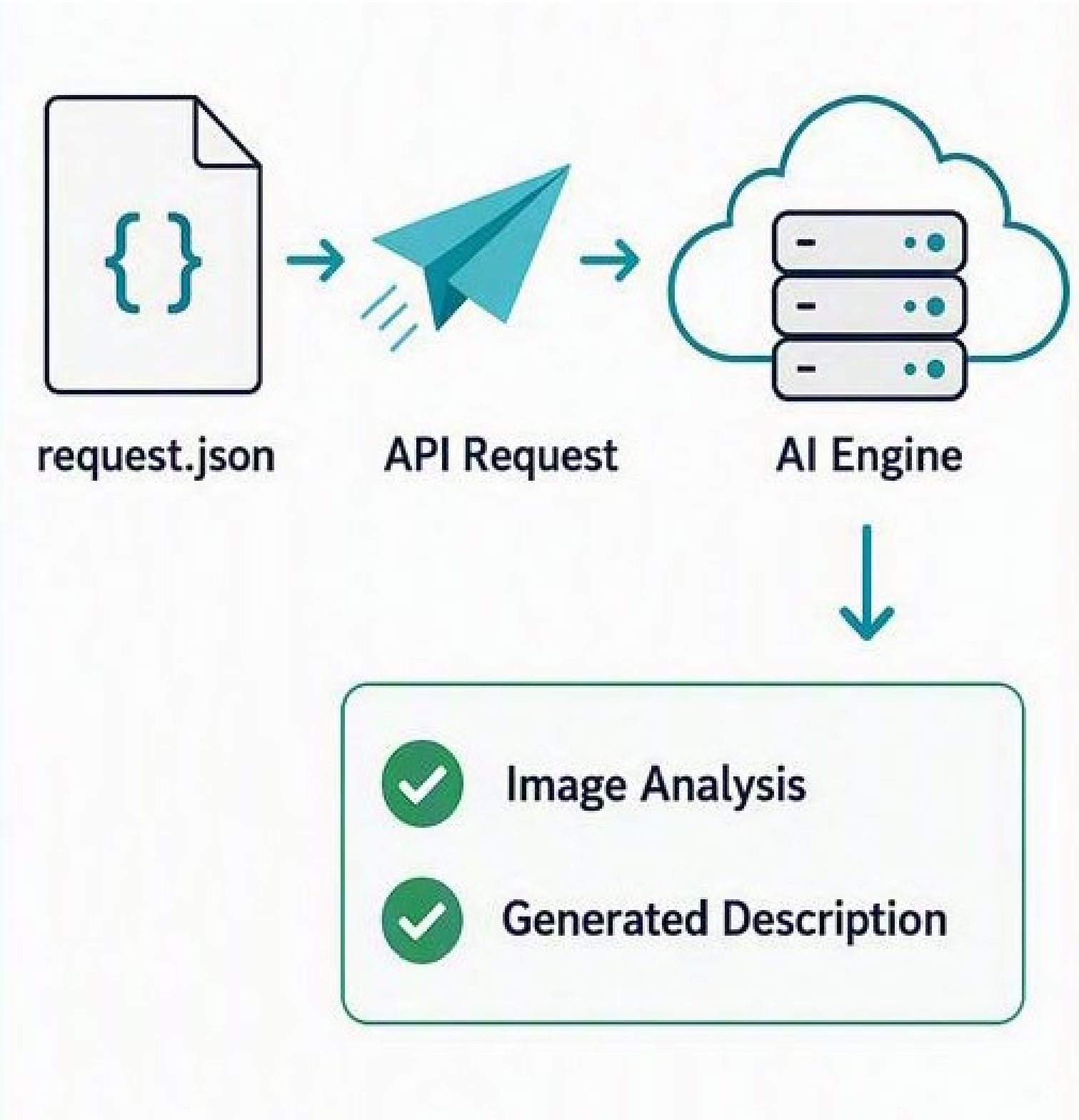
Create a JSON file containing the request to be sent to the API



STEP 3:

Send Request

Specify the created request.json and send the request to the API using curl



STEP 1 | Convert images to Base64 format

```
cd ~/03_Multimodal/  
  
base64 -w 0 flower.jpg > flower.b64
```

1

flower.b64

Save the base64 string of the image to flower.b64

```
cat flower.b64
```

2

View the contents of the Base64 file

You will see a very long text string!



```
/9j/4AAQSkZJRgABAQEAAkQAAD/4QA6RXhpZgAATU0AKgAAAAgAA1EQAAEAAAAABAQAAAFERAAQAAABAAAAAFESAAQAAABAAAAA  
wUDAwMDAwYEBAMFBwYHBwcbGwICQsJCAgKCAcHCg0KCgsMDAwMBwkODw0MDgsMDAz/2wBDAQICAgMDAwYDAwYMCAcIDAwMDAwMDA  
wMDAwMDAwMDAwMDAwMDAwMDAz/wAARCAHMAWYDASIAAhEBAxEB/8QAHwAAAQUBAQEBAQEAAAAAAAAAAAECAwQFBgcICQoL/8QAtRAAAgE  
hMUEGE1FhByJxFDKBkaEII0KxwRVS0fAkM2JyggkKFhcYGRolJicoKSo0NTY3ODk6Q0RFRkdISUpTVFVWV1hZWmNkZWZnaGlqc3R1dnd4  
pKWmp6ipqrKztLW2t7i5usLDxMXGx8jJytLT1NXW19jZ2uHi4+TL5ufo6erx8vP09fb3+Pn6/8QAHwEAAwEBAQEBAQEBAQAAAAAAAAECA  
AcFBAQAAQJ3AAECAxEEBSExBhJBUQdhcRMiMoEIFEKRobHBCSMzUvAVYnLRChYkNOEl8RcYGRomJygpKjU2Nzg50kNERUZHSElKU1RVVl  
OEhYaHiImKkpOUlZaXmJmaoqOkpaaqKmqsR00tba3uLm6wsPExcBHyMnK0tPU1dbX2Nna4uPk5ebn60nq8vP09fb3+Pn6/9oADAMBAAI  
QV6V4ihxdnfvXn3xQsi2kykDnGK80WqPYWkjinvhPeC8fpyWr0bxYv2TSge22uB+DemGKdMjvXpHje0+02scQGdw5xXFOKR2Qnd6nknha  
AV7ZrNgulW277oUZNFpfxX8R/wBsa9IqnKocCpwbzbnOVXiuW7MDQsTX8Y7Bq+gPhvFssso/ok+etCl8vUI/TdX0J8NjN9pF9K9hvU8tLQ7  
bbj0raBhURz1wP3hoVfkxUky5lNKqfLW62MCewTLitVF4qhp8fzCtRE4pomRC68e9MIzViSPmo2TaalgiMCnou01KFAqRV5pFDolqzCtR  
FSIuKALSKv50ACrnipUXHFiqVIqd0BUsaQuM9qQp7VKEoK1BZDs5pwT8ak2YFKVwM0BYiC5PSL2ZNPCn0pwSgqxDs9qUJz0xUuygR0CI/  
o5oMfpQIqNF7U1oatsnFNMMW6gZTaD2pjQ4FXWi9qY0NK40ig8FQvBx0rRkgqJ4KTGjLkt+aKuvb4PIzRQBR8RQYvsH1rLfH2k+fpj/L1F  
oHC/DHTPJuhx0NdvqsYN3820BWR4I0/ybwDG0a0vEc+y8kAHQVylnozporU86+Murrp2g3LKcHBFfK2p3rXmpSMTnJNfQvx3nZtFk7ZNf  
Y6tX0N8LkxYxnHYV89adGV10IejV9FCxSd0j/3RXqHndTvdFGLpa6C5P8Aox9q57SOLkVvz/8Aht+FawZjURjSnEhqRFBWmSD5zXV+CN  
LKwt19X2tuxz06EGjFYunhqLrVXZL+v6+8xpUnUlyow90XJU1qomKwFH3xF/tX9pXUl9Kk0rRPEMS6nbRMmyK2klWXYnoGy5CdsHGMV  
9a58sz0ljKEK0NHJXt1Xf7u4q9PkqSp72bX9epK6iomXmrLLg1EVyFsvQaM0RKnNSouDSqlPRakZJCtWY1wBTII+KsxLSAFGBUilShBTL  
yR1IEyfagpDVTil8vipVXilKZNQyiEJmjZn6VKIwaUJjtSGiHy6ALS7BRtFBRGE9BTgmaef9qUJQIjCUuzISFSbKdtzWKEJjxTdnTVgr  
YaQw57VZEeR6UNGStKw7LNoe0BUUKHtV548f4VG00R0pNFIZpIMnmirrW+6ikFyh4rh23o+tV9VthLpo001aPi6ELEDPXNMuIQ+nj6Vwx  
Y8dq2tPg2XB96x/EGlvcRz0ASuccdq5MVNRi2zqwkHKSSPE/j7eBPD5AI3M1eJ2EBklGR14r2H9oGIwwxox4JzXmFjEiYI7GqW/dXHjr
```

STEP 2 | Exporting JSON files

```
cat > request.json <<EOF
{
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "messages": [
    {
      "role": "user",
      "content": [
        { "type": "text", "text": "Please describe this image briefly."
      },
      {
        "type": "image_url",
        "image_url": {
          "url": "data:image/jpeg;base64,$(cat flower.b64)"
        }
      }
    ]
  ],
  "temperature": 0.7,
  "max_tokens": 2000,
  "stream": false
}
EOF
```



Write the following content to **request.json** file



\$(.....)

Command substitution. Embed the execution results of commands inside here.

cat flower.b64

Read the contents of flower.b64

Shorten curl arguments and pass request bodies via files

STEP 3 | Curl request with file specification

```
curl -X POST "https://api.ai.sakura.ad.jp/v1/chat/completions" ¥  
  -H "Authorization: Bearer ${AI_ENGINE_TOKEN}" ¥  
  -H "Content-Type: application/json" ¥  
  --data-binary @request.json | json_pp
```



--data-binary

Send the file contents as the request body

@request.json

request.json Load the file contents

Shorten curl arguments and pass request bodies via files

STEP 4 | See the response

```
{
  "choices": [
    {
      "finish_reason": "stop",
      "index": 0,
      "message": {
        "content": "A vibrant bouquet of mixed flowers, including dahlias, roses,
lilies, and hydrangeas, is arranged in a frosted glass vase on a wooden table. The
scene is illuminated by warm, golden light, likely from a setting sun, which creates
a soft glow and highlights the rich colors of the blooms.",
        "role": "assistant"
      }
    }
  ],
  "created": 1788380343,
  "id": "chatcmpl-8dce664d4784b0f1",
  "model": "preview/Qwen3-VL-30B-A3B-Instruct",
  "object": "chat.completion",
  "system_fingerprint": "vllm-0.26.0-tp2-6ff63b19",
  "usage": {
    "completion_tokens": 65,
    "prompt_tokens": 170,
    "total_tokens": 235
  }
}
```



content

The content of the image is explained.

(Advanced Chapter) Hands-On 4

Exploring Diverse Image Analysis Tasks

Try Image Analysis with Your Own Images

- Upload **handwritten notes, complex diagrams, or photos** and evaluate whether the model can recognize and explain them as expected.

! Important Notes

- The image file name is specified in the code. Replace it with the name of your target image file.
- Store the image file in ~/03_Multimodal.
- Images are often large. Use the JSON file approach from Hands-On 3 to include the request payload.
- **Do NOT use images containing personal or confidential information.**



Example Image Analysis Challenges

Challenge 1: Text Extraction from Signs and Receipts (OCR Accuracy Test)

Use tilted receipts or signboards with complex backgrounds and evaluate whether the VLM can accurately extract text and structure the output into JSON (e.g., item names and prices).

Task 2: Counting Multiple Objects (Recognition Limit Test)

Provide images containing many objects, such as fruits or vehicles, and test how accurately the VLM can count them.

Assignment 3: Summarizing Handwritten Notes and Whiteboards (Content Understanding Test)

Provide a photo of handwritten notes or a whiteboard and evaluate whether the VLM can accurately summarize the content and extract action items.

Key Practical Points (Cost Prompts)

Image Resolution and Cost

The higher the resolution of the image, the more grids (patches) it is divided internally, causing a **sharp increase in the number of vision tokens consumed**

- Increase in latency (processing wait time)
- Rising API usage fees

* Re-resizing to the appropriate resolution in advance according to the intended use is effective.

Vision Prompting Innovations

Instead of vaguely showing the entire image, **explicitly prompting "where to focus"** greatly improves recognition accuracy

Please analyze the second row of the table in the upper-right corner of the image.

What is the cause of the error message highlighted in red?

Optional: Development Hands-On



Try a Coding-Focused Model

Try other public preview models.

Use a coding-specialized model to upload wireframes or screenshots and generate HTML/CSS code automatically.

Example coding-capable models

- Qwen3.6-35B-A3B
- gemma-4-31B-it
- Please keep **Kimi usage to a minimum to avoid excessive load.**



Combine Multiple AI APIs

Combine the APIs you learned in previous sessions, such as:

Whisper (speech recognition)

RAG (Retrieval-Augmented Generation)

Multimodal AI

and build a practical AI application.

Summary

Summary

- ✓ We have confirmed how to handle not only text input but also input including images via API.
- ✓ Images can be included not only in URL format but also directly in requests as Base64 format.
- ✓ Multimodal models handle multiple forms of information in an integrated manner, enabling reasoning and response generation based on them.

Summary

Thank you for your hard work. They are what you have learned in this session.



You have understood the concept and basic mechanisms of multimodal



I learned how to send images and was able to actually perform image recognition.



I was able to perform the method of sending images in JSON file format.



Material for Considering How to Utilize Multimodal AI

After this, with a break, we proceed to **Module 4.**

The tokens and environment variables you created will be used as-is in the classes that follow. Don't erase it.